Advanced Day 5 — Session 5 (Hours 17–20)

GUI Fundamentals, Layout and Usability, Form Validation, and Record List UI with Tkinter

Session 5 Overview

Topics Covered Today

- Hour 17: GUI fundamentals — event loop, widgets, callbacks, service wiring
- Hour 18: Layout, usability, and resizing with nested frames and grid weights
- Hour 19: Forms + validation feedback patterns and UI/service responsibility
- Hour 20: Record list UI with Treeview/Listbox, selection callbacks, safe refresh

Source Alignment

- `Advanced/lessons/lecture/Day5_Hour1_Advanced.md` → `# Day 5, Hour 1: GUI Fundamentals with Tkinter and ttk`
- `Advanced/lessons/lecture/Day5_Hour2_Advanced.md` → `# Day 5 Hour 2 Advanced – Tkinter Layout, Usability, and Resizing`

Session 5 Outcomes

- Build event-driven Tkinter apps that stay alive via `mainloop()`
- Use nested frames, sticky, and weights for readable, resizable layouts
- Validate form input before saving and surface errors clearly to users
- Display records in a Treeview, respond to selection, and refresh safely
- Apply one consistent design rule: **GUI handles interaction; service handles rules**

Scope Guardrails for Today

In Scope

- Tkinter/ttk window, widget, callback, and layout fundamentals
- Frame decomposition, sticky behavior, and resize weight allocation
- Required-field validation, inline error feedback, and Save button state
- Treeview record browsing, selection callbacks, and safe refresh sequence

Not Yet

- Tkinter threading or async GUI patterns
- Database or file persistence backends
- Full MVC/MVP framework architecture
- Advanced sorting, filtering, or multi-level Treeview trees

Hour 17: GUI Fundamentals with Tkinter and ttk

Learning Outcomes

- Explain what makes GUI programming event-driven
- Describe the role of `mainloop()` in a Tkinter app
- Create a window with `Label`, `Entry`, and `Button` using `ttk`
- Wire a button callback to a service layer
- Show user feedback with a status label and `messagebox`

The GUI Mental Model and `mainloop()`

- Command-line programs run **top to bottom**; GUIs **wait for events**
- The user's clicks, key presses, and text entry drive execution
- Our code registers **callbacks**; the toolkit calls them when events fire

```
root = tk.Tk()
# ... create widgets and register callbacks ...
root.mainloop()   # starts the event loop; keeps window alive
```

- Without `mainloop()` the window may flash and disappear
- When the user closes the window, `mainloop()` exits

> **Key message:** The GUI handles interaction; the service layer handles rules.

The Three-Part GUI Pattern

```
widgets collect input
        ↓
  callback responds
        ↓
 service validates and stores
        ↓
  UI shows success or error
```

- **Widget** — visible UI element (Label, Entry, Button)
- **Callback** — function wired to an event via `command=self.on_add` (not `command=self.on_add()`)
- **Service layer** — ordinary Python that owns business rules and validation
- Blocking calls (sleep, network, large files) in a callback **freeze the GUI**

Demo: Service First — ContactService

```python
class ContactService:
    def __init__(self) -> None:
        self._records: list[ContactRecord] = []

    def add_record(self, name: str, email: str) -> ContactRecord:
        clean_name = name.strip()
        clean_email = email.strip().lower()
        if not clean_name:
            raise ValidationError("Name is required.")
        if "@" not in clean_email:
            raise ValidationError("Email must include '@'.")
        record = ContactRecord(name=clean_name, email=clean_email)
        self._records.append(record)
        return record
```

- Service does **not** know about widgets, `StringVar`, or `messagebox`
- Rules belong here whether input comes from GUI, CLI, or API

Demo: ContactApp — Widgets and Callback

```python
class ContactApp:
    def __init__(self, root: tk.Tk, service: ContactService) -> None:
        self.root = root
        self.service = service
        self.name_var = tk.StringVar()
        self.email_var = tk.StringVar()
        self.status_var = tk.StringVar(value="Ready.")
        self._build_ui()

    def _build_ui(self) -> None:
        frame = ttk.Frame(self.root, padding=16)
        frame.pack(fill="both", expand=True)
        ttk.Label(frame, text="Name").pack(anchor="w")
        ttk.Entry(frame, textvariable=self.name_var).pack(fill="x", pady=(0, 10))
        ttk.Label(frame, text="Email").pack(anchor="w")
        ttk.Entry(frame, textvariable=self.email_var).pack(fill="x", pady=(0, 10))
        ttk.Button(frame, text="Add Record", command=self.on_add).pack()
        ttk.Label(frame, textvariable=self.status_var).pack(anchor="w")
```

Demo: The `on_add` Callback

```
    def on_add(self) -> None:
        try:
            record = self.service.add_record(
                self.name_var.get(), self.email_var.get()
            )
        except ValidationError as exc:
            self.status_var.set(f"Error: {exc}")
            messagebox.showerror("Validation Error", str(exc))
            return
        self.status_var.set(f"Added {record.name}.")
        messagebox.showinfo("Success", f"Saved {record.name}.")
        self.name_var.set("")
        self.email_var.set("")

def main() -> None:
    root = tk.Tk()
    service = ContactService()
    ContactApp(root, service)
    root.mainloop()
```

Lab: Hello GUI and Add Form (Hour 17)

Time: 26 minutes

Tasks

- Part A — Hello GUI: create `tk.Tk()` window with a `ttk.Label`; confirm `mainloop()` keeps it alive
- Part B — Add Form: build a two-field form with `Label`, `Entry`, `Button`, and status label
- Part C — Wire a service: validate through a service; show `messagebox` on success and error
- Part D — Test all cases: blank fields, one field only, two valid fields, close and relaunch

Completion criteria

Common Pitfalls (Hour 17)

⚠️ Business logic placed inside the button callback instead of a service layer
⚠️ Forgetting `root.mainloop()` — window flashes and disappears
⚠️ Writing `command=on_add()` — calls the function at build time instead of wiring it
⚠️ Creating a widget but not calling `.pack()` or `.grid()` — widget is invisible
⚠️ Mixing `.pack()` and `.grid()` in the **same parent container** — causes layout errors
⚠️ No visible feedback — user cannot tell whether Add succeeded or failed

Quick Check: What does `mainloop()` do? Why is `command=self.on_add` correct while `command=self.on_add()` is a bug?

Hour 18: Layout, Usability, and Resizing

Learning Outcomes

- Explain parent-child geometry ownership
- Use nested frames to separate layout regions
- Configure row and column weights so a window expands predictably
- Apply sticky and padding for alignment and readability
- Identify and fix overlap, collapse, and cramped spacing defects

Parent-Child Geometry Ownership

- A Tkinter layout is a **tree** — every widget has exactly one parent
- Use **one geometry manager** for direct children of any one parent
- You may use different managers in *different* nested parents — not mixed in the **same** parent

> "Think of each frame as a room. Inside one room, choose one rule for placing furniture."

Frame decomposition pattern:

- Page-level frame holds region frames
- Each region frame manages its own local grid
- Local coordination keeps layout reasoning simple

Frame Decomposition Code Example

```
root.columnconfigure(0, weight=1)
root.rowconfigure(0, weight=1)

page = ttk.Frame(root, padding=12)
page.grid(row=0, column=0, sticky="nsew")
page.columnconfigure(0, weight=3)   # content region — wider
page.columnconfigure(1, weight=2)   # sidebar region — narrower
page.rowconfigure(1, weight=1)      # row 1 absorbs vertical growth

content = ttk.Frame(page)
content.grid(row=1, column=0, sticky="nsew")

sidebar = ttk.Frame(page)
sidebar.grid(row=1, column=1, sticky="nsew")
```

- Regions are **local layout jurisdictions** — each configures its own weights and sticky

Sticky, Weights, and Padding

Sticky — controls where a widget attaches inside its grid cell

- `ew` — horizontal stretch, `ns` — vertical stretch, `nsew` — full coverage

Weights — allocate the resize budget

- `columnconfigure(1, weight=1)` — column 1 absorbs extra width
- A column with `weight=0` (default) receives no extra space

```
content.columnconfigure(1, weight=1)
ttk.Entry(content).grid(row=0, column=1, sticky="ew", pady=(0, 8))
```

Padding — usability signal, not decoration

- Consistent `padx` / `pady` creates visual rhythm and scan paths
- Use `minsize` on `root` to protect baseline readability

Pitfalls — Missing Sticky, Missing Weights, Mixed Managers

```
# Defect: entry stays fixed while window grows (missing sticky + weight)
ttk.Entry(frame).grid(row=0, column=1)
# Fix:
frame.columnconfigure(1, weight=1)
ttk.Entry(frame).grid(row=0, column=1, sticky="ew")

# Defect: listbox does not grow vertically (missing row weight + sticky)
tk.Listbox(sidebar).grid(row=1, column=0)
# Fix:
sidebar.rowconfigure(1, weight=1)
tk.Listbox(sidebar).grid(row=1, column=0, sticky="nsew")

# Defect: mixing pack and grid in the same parent causes errors
# Fix: choose one manager per parent; use a nested frame to isolate
```

Lab: Refactor a Rough Layout (Hour 18)

Time: 26 minutes

Tasks

- Start from a provided rough layout — no weights, no sticky, no nested frames
- Sketch region decomposition in comments before coding
- Apply structural frames, alignment with sticky, and consistent padding
- Configure `rowconfigure` / `columnconfigure` weights where growth is expected
- Run the mandatory resize test: narrow, wide, and tall
- Peer-review: one commendation + one targeted adjustment

Completion criteria

✓ UI is readable and aligned
✓ Resizing is acceptable under all three test scenarios

Common Pitfalls (Hour 18)

⚠️ Missing `sticky` — entry stays tiny while its cell enlarges (overlap feel)
⚠️ Missing row/column weight — listbox or text area does not grow (collapse feel)
⚠️ Missing padding — labels, entries, and buttons visually blend (crowded feel)
⚠️ Mixed geometry manager in one parent — runtime warnings and unpredictable layout
⚠️ Uneven column weights — one region stretches too much; content area stays cramped
⚠️ Testing only at default size — a layout not tested under resize is not finished

Quick Check: A column has `weight=1` but the entry still does not stretch. What else is missing?

Hour 19: Forms and Validation Feedback Patterns

Learning Outcomes

- Validate input **before** saving — normalize first, validate second
- Implement required-field checks and actionable inline feedback
- Move focus to the first invalid field automatically
- Disable and re-enable the Save button correctly on all code paths
- Surface service-layer failures without exposing stack traces to users

Inline Validation vs Messagebox

Inline validation — preferred for field-level, expected input mistakes

- Keeps user attention in context; connects each error to the exact field
- Supports correction flow without modal interruption

Messagebox — reserved for major state changes, destructive actions, or severe failures

- Forces acknowledgement when safety matters
- Should never show a raw Python traceback

> **Rule:** Field problem → inline message + focus move. App-level failure → concise status or controlled popup.

Actionable Error Messages and Save Button Behavior

Bad messages: "Invalid input." | "Error." | "Email wrong."

Good messages:

- "Name is required. Use at least 2 letters."
- "Enter an email like name@example.com."
- "Role is required before saving."

Save button disable/enable:

```
def on_save(self) -> None:
    self._clear_inline_errors()
    self.save_btn.state(["disabled"])       # disable immediately

    try:
        result = self.service.validate_and_save(...)
    except ServiceSaveError:
        self.status_var.set("Could not save. Please try again.")
```

UI / Service Responsibility Boundary

UI layer owns:

- Reading field values from widgets
- Clearing old error state at the start of each save
- Disabling/enabling Save control
- Rendering inline errors and summary messages
- Focus management

Service layer owns:

- Input normalization — trim, lowercase, collapse whitespace
- Validation rule decisions
- Persistence call
- Raising controlled exceptions for infrastructure issues

Demo: validate_and_save and Inline Error Application

```python
def validate_and_save(self, name: str, email: str, role: str) -> ValidationResult:
    clean_name = " ".join(name.strip().split())
    clean_email = email.strip().lower()
    errors: dict[str, str] = {}
    if not clean_name:
        errors["name"] = "Name is required."
    if not clean_email:
        errors["email"] = "Email is required."
    elif "@" not in clean_email:
        errors["email"] = "Enter an email like name@example.com."
    if errors:
        return ValidationResult(ok=False, errors=errors)
    self._records.append(Contact(clean_name, clean_email, role.strip()))
    return ValidationResult(ok=True, errors={}, record=...)
```

```python
    if not result.ok:
        for field, msg in result.errors.items():
            self.error_vars[field].set(msg)  # inline label under field
        self._focus_first_invalid(result.errors)
```

Lab: Add Validation Behavior (Hour 19)

Time: 26 minutes

Required deliverables

- Required-field checks with whitespace normalization (trim before check)
- Inline error labels rendered under each invalid field
- Focus moved automatically to the first invalid field
- Save button disable/enable on all paths, including exception path (use `finally`)
- User-friendly service failure message — no stack trace shown in the UI
- Demonstrable valid save path that clears the form

Acceptance tests to run before calling lab complete

- Blank fields, spaces-only fields, malformed email, valid input, simulated service failure

Common Pitfalls (Hour 19)

⚠️ Validation rule duplicated in both UI and service — inconsistent outcomes
⚠️ Old inline errors remain visible after a successful save
⚠️ Save button stays disabled forever — `finally` block missing
⚠️ Spaces-only input passes required check — normalize **before** checking emptiness
⚠️ Silent failure — button click does nothing and shows nothing to the user
⚠️ Raw stack trace surfaced in a UI label or messagebox — translate to a calm user message

Quick Check: Give a good validation message and a bad one for an email field. What makes the good one better?

Hour 20: Record List UI with Treeview, Selection, and Safe Refresh

Learning Outcomes

- Display records in a Listbox or Treeview widget
- React to user selection and show record details in a details pane
- Choose between Listbox and Treeview for a given use case
- Implement selection callbacks that do not crash on empty selection
- Refresh list data safely using the clear → repopulate → handle-selection sequence

Listbox vs Treeview

Listbox is best when:

- Each row is a single display string
- Fields can compress into one readable line
- Column structure is not needed

Treeview is best when:

- Records have multiple fields and table-like readability matters
- Named column headers improve clarity
- Later sorting or richer row identity may be added

> **Default choice for multi-field records:** Treeview — columns match fields directly.

Quick ask: Given records with id, name, email, and role, which widget gives better clarity?

Selection Callbacks — Safe Handling

```python
self.tree.bind("<<TreeviewSelect>>", self.on_select)
```

```python
def on_select(self, _event: tk.Event) -> None:
    selected_items = self.tree.selection()
    if not selected_items:                # guard: nothing selected
        self.clear_details("No record selected")
        return
    try:
        selected_id = int(selected_items[0])   # guard: parse safely
    except ValueError:
        self.clear_details("No record selected")
        return
    record = self.service.get_record_by_id(selected_id)
    if record is None:                    # guard: record may be gone
        self.clear_details("No record selected")
        return
    self.show_details(record)
```

Refresh Safety Sequence

The three-step sequence — order is non-negotiable:

```
def refresh_records(self) -> None:
    existing = self.tree.get_children()
    if existing:
        self.tree.delete(*existing)        # step 1: clear existing rows

    records = self.service.list_records()
    for item in records:                   # step 2: repopulate from source
        self.tree.insert("", "end", iid=str(item.record_id),
                         values=(item.name, item.email, item.role))

    if not records:                        # step 3: handle selection state
        self.clear_details("No records available")
        return
    self.clear_details("No record selected")
```

Skipping step 1 causes **stale rows to stack** on subsequent refreshes.

Demo: Treeview Record Browser — Setup and Wiring

```
self.tree = ttk.Treeview(
    main, columns=("name", "email", "role"),
    show="headings", selectmode="browse", height=12,
)
self.tree.heading("name", text="Name")
self.tree.heading("email", text="Email")
self.tree.heading("role", text="Role")
self.tree.column("name", width=180, anchor="w")
self.tree.column("email", width=240, anchor="w")
self.tree.column("role", width=130, anchor="w")
self.tree.bind("<<TreeviewSelect>>", self.on_select)
```

- `show="headings"` hides the default tree column; shows only named columns
- `iid=str(item.record_id)` lets us look up the source record by ID on selection
- Details pane uses `tk.StringVar` bound to `ttk.Label` widgets for live updates

Demo: Three Mandatory Acceptance Checks

Run all three in front of students — any failure means the feature is not complete:

No-selection check

- Ensure nothing is selected → no exception; details pane shows "No record selected"

Empty-data check

- Clear source, then refresh → list shows 0 rows; details shows "No records available"; no crash

Repeated-refresh check

- Refresh three consecutive times → row count remains correct; no duplicate or stale rows

> "If your class implementation does not pass these three checks, it is not complete.

Lab: Build List-and-Details UI (Hour 20)

Time: 26 minutes

Tasks

- Scaffold a two-region layout — list area (Treeview recommended) + details pane with fallback text
- Load in-memory records into the Treeview on startup
- Bind `<<TreeviewSelect>>` and populate the details pane on valid selection; guard empty selection
- Implement `refresh_records()` with the exact clear → repopulate → handle-selection sequence
- Run all three acceptance checks (no-selection, empty-data, repeated-refresh)

Completion criteria

Common Pitfalls (Hour 20)

⚠️ Selection callback crashes when `tree.selection()` returns an empty tuple — guard it first
⚠️ Skipping the clear step in `refresh_records` — old rows stack on top of new rows
⚠️ Details pane shows stale content after source data is cleared and refreshed
⚠️ `iid` stored as an integer — Treeview requires a string `iid`; type mismatch crashes lookup
⚠️ Refresh sequence out of order — repopulate before clear causes visible duplicates
⚠️ Optional search/filter added before core acceptance checks pass — stabilize core first

Quick Check: What can go wrong if you refresh without clearing existing rows? What must happen when no row is selected?

Session 5 Wrap-Up

What We Built Today

- A working event-driven Tkinter GUI with service-layer separation (Hour 17)
- Intentional nested-frame layouts with predictable resize behavior (Hour 18)
- Form validation with inline feedback, focus management, and safe Save state (Hour 19)
- A record browser with Treeview, selection callbacks, and duplicate-free refresh (Hour 20)

The One Design Rule That Governed All Four Hours

"The GUI handles interaction; the service layer handles rules."

Day 5 Homework and Study Checklist

- Re-run your Hour 17 app and add a third field with its own validation rule
- Apply three-step frame decomposition to your Hour 18 layout and run all resize tests
- Confirm your Hour 19 form passes all five test cases: blank, spaces, malformed email, valid, service error
- Run the three acceptance checks on your Hour 20 record browser: no-selection, empty-data, repeated-refresh
- Optional: add a search/filter field to the Treeview — only after all core checks pass

Next Session Preview

Session 6 (Hours 21–24)

- Connecting the GUI to file or SQLite persistence
- Edit and delete record workflows with confirmation dialogs
- More complex layouts — scrollable frames and status bars
- Checkpoint: integrated GUI application milestone

Thank You!

Save your work.

Commit the latest good state.

Be ready to wire persistence tomorrow.